

Project Description for Infosys Interns

Project Title: Online Exam Monitoring & Integrity Analytics Platform

Project Statement: This app is a Python-based online examination monitoring platform designed to facilitate secure and transparent assessments for educational institutions. Built using Flask and OpenCV, the system assists invigilators in monitoring examination sessions through webcam-based face presence tracking, browser activity logging, and integrity-focused event recording. A rule-based engine computes candidate integrity scores based on session behaviour, while a LangChain AI agent automatically generates natural language integrity summary reports for invigilator review. Data Science driven analytics provide corpus-level insights into session behaviour patterns, event distributions, and risk profiling across the examination cohort. Rather than automated disciplinary actions, ExamGuard equips invigilators with structured evidence, intelligent summaries, and meaningful analytics to ensure fair and consistent assessment reviews.

Submitted by,

KANISHKA S

VIT VELLORE

1. Abstract

ExamGuard is a web-based online examination verification system developed to provide a structured pre-examination verification process for candidates. The system is implemented using Python and Flask for backend application development, HTML for the user interface, and MySQL for storing candidate information. It provides candidate registration and login functionality followed by sequential camera, microphone, and selfie verification stages before granting access to the dashboard. The system organizes these verification activities into a simple and user-friendly workflow, allowing candidates to complete the required checks before proceeding to the examination environment. The modular architecture provides a foundation for integrating advanced examination monitoring and proctoring capabilities in future versions. Overall, ExamGuard demonstrates how web technologies and database connectivity can be combined to establish a basic examination verification platform.

2. Introduction

The rapid adoption of digital learning and remote assessment has increased the need for reliable online examination platforms. Unlike conventional examinations conducted in controlled physical environments, online examinations require candidates to access assessments through personal computing devices and network connections. This creates a requirement for preliminary verification of essential device capabilities and candidate identity-related information before an examination begins. A structured verification process can help ensure that the candidate's system is appropriately prepared and that the required input devices are available.

ExamGuard is designed as a web-based application that organizes the candidate's entry process into multiple stages. The application begins with candidate registration and authentication, allowing users to create an account and subsequently access the system using their credentials. After successful login, the candidate is directed through camera, microphone, and selfie verification pages in a predefined sequence. This approach provides a clear workflow in which each stage represents a specific preparation or verification activity before the candidate reaches the application dashboard.

The project focuses primarily on establishing the core software structure required for an online examination verification environment rather than implementing a complete automated proctoring system. Flask manages application routing and communication between the interface and database, while MySQL stores candidate records. The modular design allows additional capabilities such as automated face verification, examination management, activity monitoring, reporting, and stronger authentication mechanisms to be incorporated later. Thus, the current implementation serves as a functional foundation for developing a more comprehensive examination management and monitoring platform.

3. Problem Statement

Traditional online examination systems may provide authentication and examination access without organizing the preliminary verification activities required before a candidate begins an assessment. Candidates may have access to a camera, microphone, or other required devices, but the system may not provide a structured process for checking these requirements before entering the examination environment.

There is therefore a need for a web-based platform that provides a systematic candidate onboarding and verification workflow. The system should allow candidates to register, authenticate themselves, complete predefined verification stages, and proceed to a controlled application area after completing those stages.

ExamGuard addresses this requirement by integrating candidate registration, login, camera verification, microphone verification, selfie verification, and dashboard access into a single Flask-based application connected to a MySQL database.

4. Objectives

The primary objectives of the ExamGuard project are:

- a. To develop a web-based online examination verification application.
- b. To provide candidate registration and login functionality.
- c. To store candidate information using a relational MySQL database.
- d. To establish a sequential verification workflow before examination access.
- e. To provide dedicated camera verification and microphone verification stages.
- f. To provide a selfie verification stage as part of candidate verification.
- g. To redirect verified candidates to a dashboard.
- h. To create a modular architecture that can support future examination and proctoring features.
- i. To demonstrate integration between a Python Flask application and MySQL database.

5. Scope

The scope of ExamGuard covers the development of the basic web application required for candidate authentication and pre-examination verification.

The current system includes:

- Candidate registration.
- Candidate login.
- Candidate information storage.
- Database connectivity.
- Camera verification page.
- Microphone verification page.
- Selfie verification page.
- Candidate dashboard.
- Navigation between verification stages.

- Flask-based server-side routing.
- MySQL-based candidate data management.

The current implementation is intended primarily as a functional project prototype and educational application. Advanced features such as automated facial recognition, real-time proctoring, examination question management, automated suspicious activity detection, result processing, and administrator management are outside the current implementation and may be considered for future development.

6. Existing System

Existing online examination platforms generally provide basic user authentication and examination access through a web browser. In some systems, candidates are required to independently verify whether their camera and microphone are functioning before beginning an examination. The verification process may vary depending on the platform and may not always be integrated into a single structured workflow.

A basic examination platform may contain separate login and examination pages without maintaining a dedicated sequence of device verification stages. Such an approach can make the pre-examination process less organized and may require candidates to manually confirm that their devices are ready.

Another limitation of a basic system is the absence of a modular foundation for extending the application with additional verification and monitoring functionality. Features such as identity verification, session monitoring, event logging, and administrative reporting may require substantial architectural changes if they were not considered during the initial application design.

7. Proposed System

ExamGuard proposes a structured web-based verification system in which the candidate progresses through predefined stages before reaching the dashboard. The system starts with registration for new candidates and authentication for existing candidates.

After successful login, the candidate is directed to the camera verification stage. The workflow then proceeds to microphone verification and selfie verification before the candidate reaches the dashboard. This sequence provides a consistent onboarding process for all candidates.

The proposed system uses Flask to manage application routes and HTML templates to provide the user interface. MySQL is used for persistent candidate data storage, while a dedicated database connection module manages communication between the Flask application and the database.

The modular structure also provides a foundation for future integration of advanced features such as face detection, examination session management, browser activity monitoring, automated reporting, and administrator dashboards.

8. Technology Stack

8.1 Python

Python is used as the primary programming language for the backend. Its simplicity and extensive library ecosystem make it suitable for developing web applications and integrating database functionality.

8.2 Flask

Flask is used as the web application framework. It manages HTTP requests, URL routes, template rendering, form processing, and application flow.

8.3 MySQL

MySQL is used as the relational database management system. Candidate information such as identification details, name, email, and password is stored in the database.

8.4 MySQL Connector

The mysql-connector-python package provides communication between the Python application and MySQL database.

8.5 HTML

HTML is used to create the web pages and user interface components of the application.

8.6 Web Browser

A standard web browser is used by candidates to access the application and interact with the verification pages.

8.7 Development Tools

The project can be developed and executed using common development environments such as Visual Studio Code or similar Python-supported IDEs.

9. System Requirements

9.1 Hardware Requirements

The application can be executed on a standard computer system with:

- Processor: Dual-core processor or above.
- RAM: Minimum 4 GB recommended.
- Storage: Sufficient space for Python, MySQL, and project files.
- Webcam: Required for camera/selfie verification stages.
- Microphone: Required for microphone verification.
- Network connectivity: Required depending on the deployment environment.

9.2 Software Requirements

- Windows, Linux, or another Python-supported operating system.
- Python.
- Flask.
- MySQL Server.
- MySQL Connector for Python.
- Web browser.
- Project source code.
- Required HTML templates.

9.3 Functional Requirements

The system shall:

- a. Allow a new candidate to register.
- b. Store candidate details in the database.
- c. Allow registered candidates to log in.
- d. Validate login credentials.
- e. Direct authenticated candidates to the camera verification page.
- f. Provide microphone verification.
- g. Provide selfie verification.
- h. Provide access to the dashboard after the verification workflow.

9.4 Non-Functional Requirements

The system should provide:

- Usability.
- Maintainability.
- Reliability.
- Appropriate response time.
- Database consistency.
- Modular implementation.
- Extensibility for future features.

10. System Architecture

ExamGuard follows a basic three-layer application structure consisting of the presentation layer, application layer, and database layer.

Presentation Layer

The presentation layer consists of HTML templates that provide the candidate-facing interface. Pages include the home page, camera verification page, microphone verification page, selfie verification page, and dashboard.

Application Layer

The application layer is implemented using Python and Flask. Flask routes receive user requests, process registration and login operations, interact with the database, and control the navigation between verification stages.

Database Layer

The database layer is implemented using MySQL. Candidate records are stored in the candidates table. The db.py module provides the database connection required by the Flask application.

High-Level Architecture

Candidate → Web Interface → Flask Application → Database

The verification flow can be represented as:

Home → Registration/Login → Camera Check → Microphone Check → Selfie Check → Dashboard

11. System Workflow

12. Module Description

12.1 Home Module

The home module acts as the entry point of the application. It provides access to registration and login functionality.

12.2 Candidate Registration Module

The registration module collects candidate information such as full name, email, and password. The information is submitted to the Flask backend and stored in the MySQL database.

12.3 Candidate Login Module

The login module receives candidate credentials and checks them against the records stored in the database. Valid credentials allow the candidate to continue to the verification workflow.

12.4 Camera Verification Module

The camera verification module provides the first verification stage following successful login. It represents the stage at which the candidate's camera readiness can be checked.

12.5 Microphone Verification Module

The microphone verification module provides the next stage in the verification sequence and is intended to ensure that the candidate can access the required audio input device.

12.6 Selfie Verification Module

The selfie verification module provides a candidate image verification stage before access to the dashboard.

12.7 Dashboard Module

The dashboard acts as the destination after completion of the verification workflow. It provides the application area that can later be extended with examination-related functionality.

12.8 Database Module

The database module is implemented through db.py. It establishes the MySQL connection used by the Flask application to access candidate information.

13. Database Design

The project uses a MySQL database named examguard.

The primary table currently used by the application is:

Candidates Table

Field	Data Type	Description
id	INT	Unique candidate identifier
full_name	VARCHAR	Candidate's full name
email	VARCHAR	Candidate's email address
password	VARCHAR	Candidate password

The id field functions as the primary key and is automatically incremented for new records.

The database provides persistent storage for candidate registration information and allows the login module to retrieve candidate records during authentication.

The database design can be extended in future versions with additional tables for examination sessions, verification results, event logs, examination attempts, results, and administrator records.

14. Implementation

14.1 Application Initialization

The Flask application is initialized in app.py. Flask is responsible for creating the web server and handling application routes.

14.2 Routing

Separate routes are provided for the major stages of the application. These routes control the transition between registration, login, camera verification, microphone verification, selfie verification, and dashboard pages.

14.3 Registration Implementation

When a candidate submits the registration form, the Flask application receives the submitted information and inserts the candidate details into the MySQL database.

14.4 Login Implementation

The login implementation retrieves the submitted candidate credentials and performs a database query to determine whether a matching record exists. A successful authentication redirects the candidate to the camera verification stage.

14.5 Database Connectivity

The db.py module contains the database connection function. This function establishes a connection between the Python application and the MySQL examguard database.

14.6 Template Rendering

Flask renders HTML templates corresponding to each application stage. This separates the user interface from the backend routing logic.

14.7 Verification Flow

The application implements a sequential flow consisting of camera verification, microphone verification, and selfie verification. Each stage is represented through a dedicated template.

14.8 Dashboard

The dashboard is displayed after the candidate completes the verification sequence and provides a foundation for integrating examination functionality in future versions.

15. Agile Development Process

The project follows an Agile-oriented development approach in which the application is developed incrementally through smaller functional components.

Instead of attempting to implement the complete system simultaneously, the project can be divided into manageable development activities such as:

- Requirement identification.
- Application structure creation.
- Database development.
- Registration implementation.
- Login implementation.
- Verification workflow development.
- Dashboard implementation.
- Testing and debugging.
- Documentation.

The iterative approach allows individual components to be implemented and tested before being integrated into the complete application.

Agile development also supports changes during development. For example, individual verification stages can be modified or expanded without requiring the entire application structure to be redesigned.

16. Sprint Planning and Execution

The project development can be organized into the following sprint structure.

Sprint 1 – Requirement Analysis and Project Setup

Activities:

- Identify system requirements.
- Define project objectives.
- Select technologies.
- Create Flask project structure.
- Configure the development environment.

Deliverable:

A basic Flask application structure.

Sprint 2 – Database and Authentication

Activities:

- Create the MySQL database.
- Create the candidates table.
- Establish database connectivity.
- Implement registration.
- Implement login.

- Test database operations.

Deliverable:

Functional candidate registration and authentication.

Sprint 3 – Verification Workflow

Activities:

- Implement camera verification route.
- Implement microphone verification route.
- Implement selfie verification route.
- Connect the verification stages.
- Test navigation between stages.

Deliverable:

Functional sequential verification workflow.

Sprint 4 – Dashboard and Integration

Activities:

- Implement dashboard.
- Connect all application modules.
- Test complete candidate workflow.
- Fix application errors.
- Perform final validation.

Deliverable:

Integrated ExamGuard application.

17. Testing Methodology

Testing was considered at multiple levels to verify that the individual modules and complete workflow function as expected.

Unit-Level Testing

Individual application functions and routes are checked to ensure that they perform their intended operations.

Integration Testing

Integration testing verifies communication between Flask routes, HTML templates, and the MySQL database.

Functional Testing

Functional testing verifies whether the system performs the expected operations, including registration, login, verification navigation, and dashboard access.

Workflow Testing

The complete candidate journey is tested from application entry through authentication and verification to dashboard access.

Validation Testing

Input and application responses are examined to identify incorrect or incomplete inputs and ensure appropriate system behavior.

18. Test Cases and Results

Test Case	Test Description	Expected Result	Result
TC01	Open application	Home page should load	Passed
TC02	Register new candidate	Candidate should be stored in database	Passed
TC03	Login with valid credentials	Candidate should proceed to camera verification	Passed
TC04	Login with invalid credentials	Login should not be accepted	Passed
TC05	Open camera verification	Camera verification page should load	Passed
TC06	Proceed to microphone verification	Microphone page should load	Passed
TC07	Proceed to selfie verification	Selfie page should load	Passed
TC08	Complete verification workflow	Candidate should reach dashboard	Passed
TC09	Database connection	Application should connect to MySQL	Passed
TC10	Dashboard access	Dashboard should be displayed after workflow	Passed

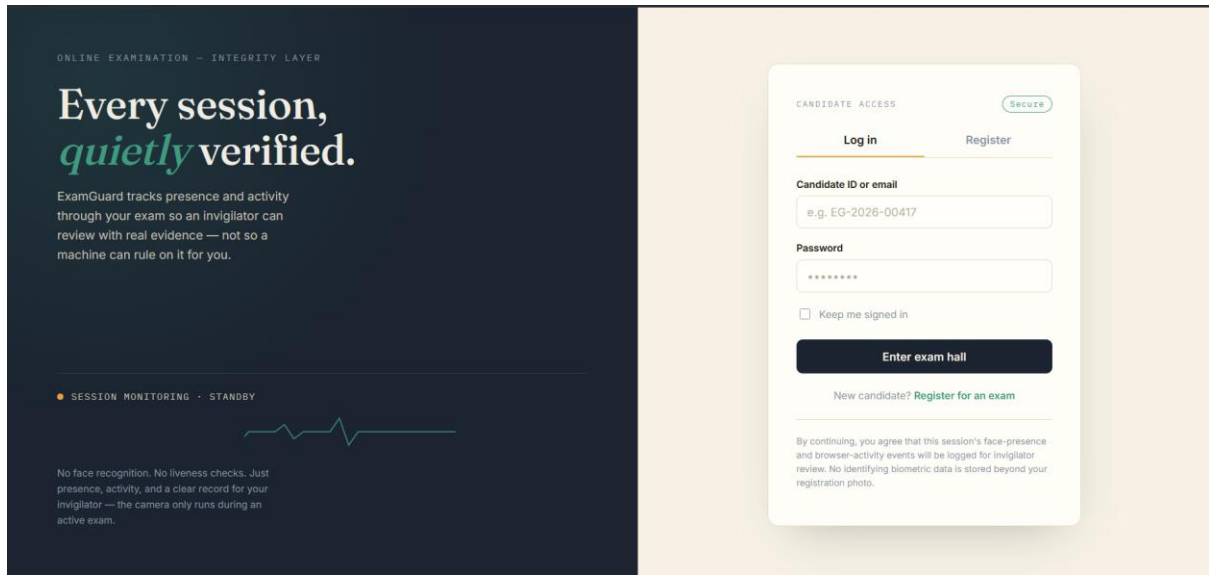
The testing process confirms the core application flow from candidate entry through authentication, verification stages, and dashboard access.

19. System Screenshots

This section should contain screenshots captured from the implemented application.

19.1 Home Page

The home page provides the primary entry point to ExamGuard and provides options for candidate registration and login.



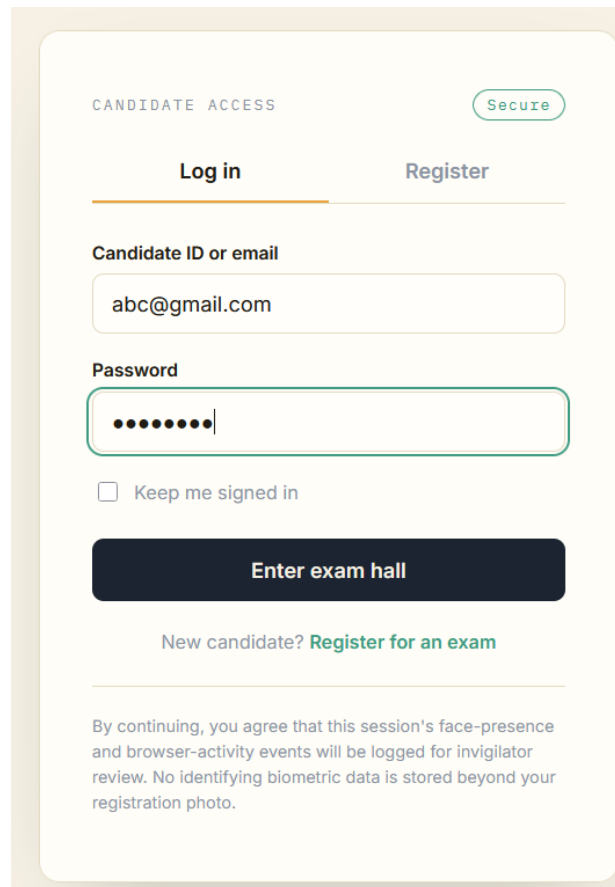
19.2 Candidate Registration

The registration interface allows a new candidate to enter the required personal and authentication information.

The image shows a 'CANDIDATE ACCESS' form with a 'Secure' badge. It has tabs for 'Log in' and 'Register'. The 'Register' tab is active. The form includes fields for 'Full name' (filled with 'Kanishka'), 'Email' (filled with 'kanishka220105@gmail.com'), and 'Password' (filled with dots). Below the password field, a note states: 'Your candidate ID is generated after this step — you'll capture your registration photo next.' There is a large 'Create candidate profile' button. At the bottom, it says 'Already registered? Log in instead' and includes a disclaimer about logging face-presence and browser-activity events.

19.3 Candidate Login

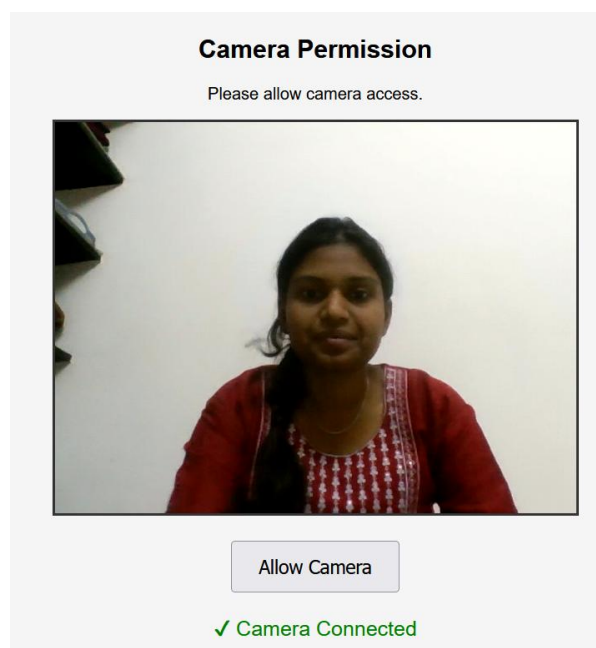
The login page allows an existing candidate to enter their credentials.



The login form is titled "CANDIDATE ACCESS" and includes a "Secure" badge. It features two tabs: "Log in" (active) and "Register". The "Log in" section contains a "Candidate ID or email" field with the value "abc@gmail.com" and a "Password" field with masked characters. Below the password field is a checkbox labeled "Keep me signed in". A large dark blue button labeled "Enter exam hall" is positioned below the checkbox. At the bottom, there is a link "New candidate? Register for an exam". A disclaimer at the very bottom states: "By continuing, you agree that this session's face-presence and browser-activity events will be logged for invigilator review. No identifying biometric data is stored beyond your registration photo."

19.4 Camera Verification

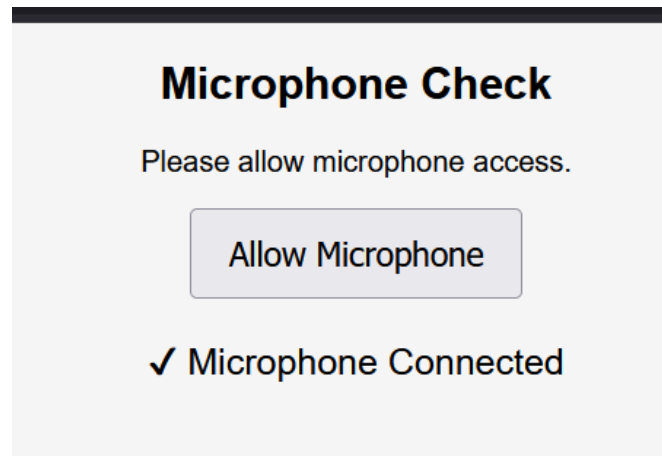
The camera verification interface represents the first verification stage after authentication.



The camera verification interface is titled "Camera Permission" and includes the text "Please allow camera access." Below this is a video feed showing a woman with dark hair wearing a red patterned top. At the bottom of the interface is a button labeled "Allow Camera" and a green checkmark followed by the text "Camera Connected".

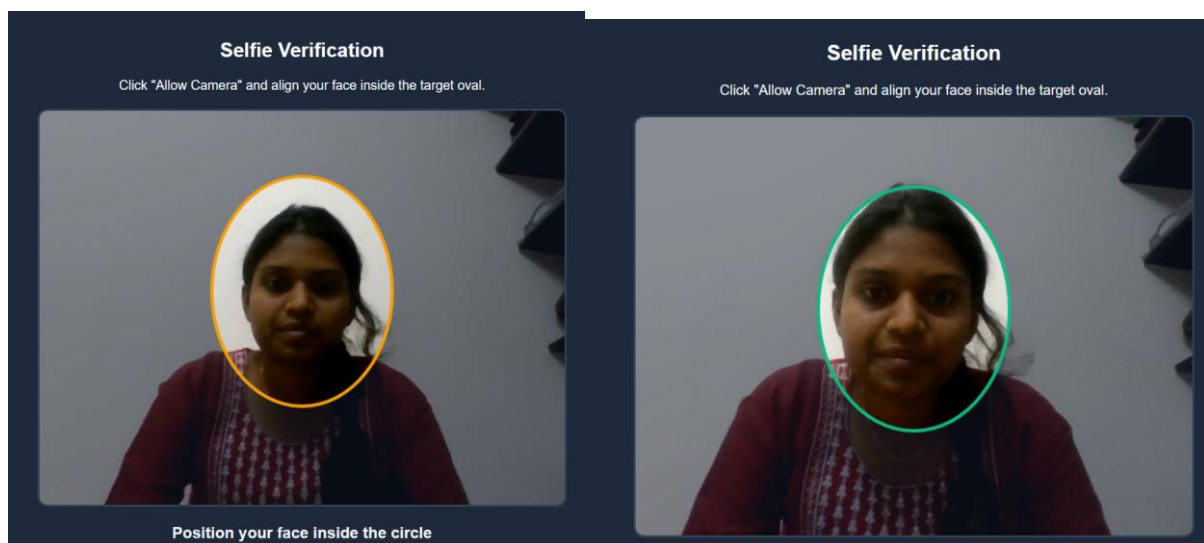
19.5 Microphone Verification

The microphone verification page provides the second device verification stage.



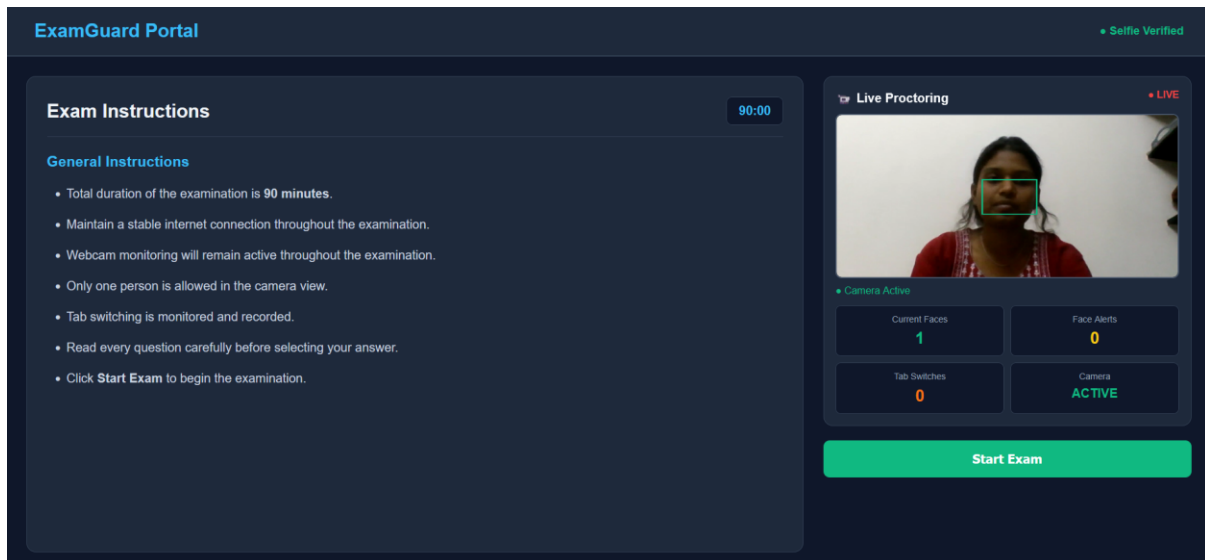
19.6 Selfie Verification

The selfie verification page provides the final verification stage before dashboard access.



19.7 Dashboard

The dashboard represents the destination after completing the verification workflow.



20. Performance Analysis

The performance of the current application can primarily be evaluated in terms of page response, database interaction, workflow continuity, and application stability.

The Flask architecture provides a lightweight backend suitable for the current prototype. Database operations are limited to candidate registration and credential verification, reducing the amount of database processing required during normal application use.

The application follows a simple sequential workflow, which reduces unnecessary navigation between unrelated pages. The modular route structure also makes individual components easier to test and maintain.

For production deployment, further performance evaluation should be conducted using multiple concurrent users, larger database records, network latency measurements, and server resource monitoring. These tests would provide a more comprehensive assessment of scalability.

21. Results and Discussion

The implemented ExamGuard system successfully establishes a structured candidate verification workflow. The application provides candidate registration and login functionality and connects these operations to a MySQL database for persistent data storage.

Following successful authentication, candidates can proceed through the camera, microphone, and selfie verification stages before reaching the dashboard. This demonstrates the integration of multiple application routes into a single sequential workflow.

The project also demonstrates the integration of Flask, HTML, and MySQL within a web application. The modular structure separates database connectivity, backend routing, and user interface components, providing a foundation for future development.

The current implementation achieves the core objectives of developing a basic online examination verification platform. However, the present system represents a foundational implementation rather than a complete automated examination proctoring solution.

22. Limitations

Although ExamGuard provides the core verification workflow, several limitations remain in the current implementation.

- Automated facial recognition is not currently implemented.
- The system does not perform advanced liveness detection.
- Real-time browser activity monitoring is not included.
- Automated suspicious activity detection is not implemented.
- Examination question and answer management is outside the current scope.
- Automated examination result generation is not included.
- The current authentication implementation requires additional security hardening for production use.
- Password hashing should be implemented before production deployment.
- Database credentials should be stored securely rather than directly in application configuration.
- HTTPS and production-grade session security should be implemented for real-world deployment.
- Administrative functionality is not currently included.
- The current implementation does not provide comprehensive examination analytics.

23. Conclusion

ExamGuard presents a basic web-based framework for organizing candidate authentication and pre-examination verification. The system integrates Flask for backend application development, HTML for the user interface, and MySQL for persistent candidate data storage. Its workflow guides candidates from registration or login through camera, microphone, and selfie verification before reaching the dashboard.

The project demonstrates the practical integration of web application development and relational database management within an online examination context. Its modular structure allows individual components to be maintained and extended without significantly changing the overall application architecture.

The current implementation provides a foundation for developing a more comprehensive examination platform. Future versions can extend the system with automated face verification, liveness detection, browser activity monitoring, examination management, secure authentication, session tracking, administrator functionality, reporting, and analytics. These enhancements can transform the current

verification prototype into a more complete online examination management and monitoring solution.